

Consultas dinámicas: Specifications y paginación

7. Consultas dinámicas: Specifications y paginación

Consultas dinámicas: Specifications y paginación

Ya sabes recuperar y modificar un objeto cuando conoces su id — `findById()` lo trae, cambias sus campos, `save()` lo guarda. Pero un listado real casi nunca busca por id: busca por criterios que el usuario elige, y además no puede devolver el catálogo entero de golpe. Este apartado resuelve dos problemas distintos, que comparten el mismo listado como escenario.

El punto de partida: buscar por un identificador

Hasta ahora has trabajado con operaciones en las que conoces el identificador exacto del objeto. Para recuperar o modificar un libro, el service utiliza `findById(id)`, obtiene una entidad concreta y trabaja directamente con ella.

Eso funciona perfectamente cuando la petición se refiere a «el libro número 42». El problema cambia cuando el usuario no conoce ningún identificador y quiere buscar «libros de menos de 20 € publicados por una editorial concreta».

La primera idea que se te podría ocurrir

Declarar un método derivado por nombre para cada filtro, como `findByPrecioLessThan(BigDecimal precio)`. Y funciona bien mientras todos los usuarios busquen siempre por lo mismo.

El problema real es que cada usuario puede querer una combinación distinta: uno busca solo por precio máximo, otro solo por título, otro por precio mínimo y máximo a la vez... y no sabes de antemano cuál de esas combinaciones va a pedir cada uno.

Problema A: los filtros fijos no escalan

En cuanto el buscador tiene varios filtros opcionales de verdad — título, precio mínimo, precio máximo, editorial — cubrir cada combinación con un método derivado distinto crece exponencialmente.

Cada filtro nuevo duplica el número de métodos

Filtros opcionales	Métodos que necesitarías
1 (solo título)	2 — findAll() y findByTituloContaining(...)
2 (+ precio mínimo)	4 — incluyendo findByTituloContainingAndPrecioGreaterThanOrEqual(...)
3 (+ precio máximo)	8 — incluyendo ...AndPrecioGreaterThanOrEqualAndPrecioLessThanOrEqual(...)
4 (+ editorial)	16 — con nombres como ...AndPrecioLessThanOrEqualAndEditorialId(...)

Y esto es solo el principio

Con solo 4 filtros ya tienes 16 combinaciones posibles, y cada filtro nuevo que añadas duplica ese número — sin contar con que un nombre de método así de largo deja de ser legible mucho antes de llegar a 16.

Las piezas nuevas, traducidas desde SQL

Tres nombres que van a aparecer juntos y conviene no confundir. Cada uno tiene su equivalente exacto en el SQL que ya conoces:

Concepto Java	Su equivalente en SQL	Qué es en la práctica
Root<Libro>	FROM libro	Representa la tabla que consultas; root.get("titulo") es "la columna titulo"
CriteriaBuilder	Los operadores de un WHERE (=, LIKE, >=...)	Un método Java por cada tipo de comparación, en vez de texto SQL
Predicate	Una condición ya completa del WHERE	Lo que construye el CriteriaBuilder, y lo que tu código debe devolver

Una Specification, en una frase

Una `Specification<Libro>` es, literalmente, una función con esta forma: recibe el `root` y el `criteriaBuilder`, y devuelve un `Predicate` — una única condición, construida con código Java en vez de escrita como texto.

Resolviendo el problema A

Vas a construir la solución completa a este problema, de principio a fin, antes de complicarla con nada más.

1. Habilitar Specifications en el repositorio

```
public interface LibroRepository extends  
    JpaRepository<Libro, Long>,  
    JpaSpecificationExecutor<Libro> {  
}
```

JpaSpecificationExecutor añade la capacidad de ejecutar Specifications sobre este repositorio, junto al save/findById/findAll que ya tenías.

2. Tu primera Specification, condición por condición

Empieza por la parte que construye el Predicate, sin la parte del filtro opcional todavía:

```
(root, query, criteriaBuilder) ->
    criteriaBuilder.like(criteriaBuilder.lower(root.get("titulo")), "%" +
titulo.toLowerCase() + "%");
```

Línea a línea, con la tabla de antes delante

- `root.get("titulo")` es la columna titulo.
- `criteriaBuilder.lower(...)` pasa esa columna a minúsculas antes de comparar (para que la búsqueda no distinga mayúsculas).
- `criteriaBuilder.like(..., "%valor%")` construye el Predicate — la condición LIKE '%valor%', pero escrita con métodos Java en vez de texto SQL.

El tercer parámetro, `query`, no lo necesitas para filtros simples como este — está ahí porque la forma de la función lo exige, pero lo dejas sin usar.

3. El filtro es opcional: `Specification.unrestricted()`

Este filtro puede no venir. Una `Specification` puede decidir, en tiempo de ejecución, si su condición entra o no en la consulta final — algo que un método derivado no podía hacer.

La clase LibroSpecifications, con un método por filtro

```
public final class LibroSpecifications {

    private LibroSpecifications() { }

    public static Specification<Libro> tituloContiene(String titulo) {
        if (titulo == null || titulo.isBlank()) {
            return Specification.unrestricted();
        }
        return (root, query, criteriaBuilder) ->
            criteriaBuilder.like(criteriaBuilder.lower(root.get("titulo")), "%" +
titulo.toLowerCase() + "%");
    }

    public static Specification<Libro> precioMayorOIgualA(BigDecimal precioMin) {
        if (precioMin == null) {
            return Specification.unrestricted();
        }
        return (root, query, criteriaBuilder) ->
            criteriaBuilder.greaterThanOrEqualTo(root.get("precio"), precioMin);
    }

    // Resto de filtros (precioMenorOIgualA, perteneceAEditorial...) – mismo patrón.
}
```

Specification.unrestricted(), qué hace de verdad

Specification.unrestricted() es una Specification especial que ya trae Spring Data: su Predicate no impone ninguna condición real, así que al combinarla con las demás es como si no existiera.

El patrón se repite en cada método

Si el filtro es...	La Specification devuelve...	Efecto en el WHERE
null (no indicado)	Specification.unrestricted()	Ninguna condición añadida — se salta de verdad
Un valor real	El Predicate correspondiente (LIKE, greaterThanOrEqualTo...)	Se añade al WHERE

4. El DTO de filtro

```
public record LibroFiltroDTO(String titulo, BigDecimal precioMin, BigDecimal precioMax,  
Long editorialId) {}
```

El listado completo, en List

```
@Transactional(readonly = true)
public List<LibroResponseDTO> findByFiltro(LibroFiltroDTO filtro) {
    Specification<Libro> spec = Specification
        .where(LibroSpecifications.tituloContiene(filtro.titulo()))
        .and(LibroSpecifications.precioMayorOIgualA(filtro.precioMin()));

    return libroRepository.findAll(spec)
        .stream()
        .map(this::mapToDTO)
        .toList();
}
```

where(...).and(...): encadenando las piezas

`Specification.where(...).and(...)` encadena las piezas en una sola consulta final — las que son `unrestricted()` no añaden nada. `libroRepository.findAll(spec)` es el `findAll` de siempre, pero con una `Specification` como argumento: sigue devolviendo todos los libros que cumplan la condición, sin ningún límite de página todavía.

5. El controller

```
@GetMapping
public ResponseEntity<List<LibroResponseDTO>> getAll (@ModelAttribute LibroFiltroDTO filtro)
{
    return ResponseEntity.ok(libroService.findByFiltro(filtro));
}
```

Problema A resuelto — pero...

@ModelAttribute mapea automáticamente los parámetros de query string (?titulo=...&precioMin=...) a los campos del DTO. Con esto, el problema A está resuelto de verdad — pero si tu tabla tiene miles de filas, esta respuesta seguiría devolviéndolas todas de golpe. Ese es el problema B.

Problema B: tampoco puedes devolver todo de golpe

Ninguna API devuelve "todo" en una sola respuesta — si una tabla tiene un millón de filas, no tiene sentido mandarlas todas de golpe. Este problema no tiene nada que ver con los filtros del problema A: pasaría igual con un `findAll()` liso y llano.

Resolviendo el problema B: paginación

Es la misma idea que cuando buscas algo en Google: con miles de resultados posibles, no te enseña todos de golpe — te da los primeros 10 y un botón para pedir los siguientes. La paginación aplica exactamente esa idea a tu API.

Página	Filas que contiene
0 (la primera)	Filas 1 a 20
1	Filas 21 a 40
2 (la última)	Filas 41 a 47

La primera página es la 0, no la 1

⚠ Error habitual

Spring Data numera las páginas empezando desde cero, igual que un índice de array. Pedir la "página 1" te da la segunda página, no la primera.

Spring Data modela todo esto con dos tipos

Tipo	Qué representa
Pageable	La petición: qué página, de qué tamaño, con qué orden — la construyes tú antes de consultar
Page<T>	La respuesta: el contenido de esa página, más metadatos (total de elementos, total de páginas...)

Por debajo, sigue siendo SQL de toda la vida

Hibernate traduce cada Pageable en las cláusulas LIMIT/OFFSET que ya conoces de SQL. Pedir la página 2 (la tercera) con tamaño 20 genera, por debajo, algo como:

```
SELECT * FROM libro ORDER BY titulo LIMIT 20 OFFSET 40;
```

LIMIT y OFFSET, explicados

LIMIT 20 dice "tráeme como mucho 20 filas"; OFFSET 40 dice "sáltate las primeras 40" — las que ya salieron en las páginas 0 y 1. Spring Data calcula ese OFFSET por ti, multiplicando el número de página por el tamaño ($2 \times 20 = 40$); tú solo indicas cuál página quieres.

Pageable no lleva ninguna anotación delante

A diferencia del `@ModelAttribute` LibroFiltroDTO filtro de antes, no hace falta: en cuanto Spring ve un parámetro de tipo `Pageable` en un método de un controller, busca automáticamente en la query string tres nombres muy concretos — `page`, `size` y `sort` — y construye el `Pageable` él solo.

Lo que ve el cliente por HTTP

Con esos tres nombres reservados, una petición como esta ya te da un Pageable completo, sin escribir un solo @RequestParam:

```
GET /api/v1/libros?page=0&size=3&sort=titulo,asc
```

La respuesta trae más metadatos de los que parece

```
{  
  "content": [ /* los libros de esta página */ ],  
  "totalElements": 7,  
  "totalPages": 3,  
  "number": 0,  
  "size": 3,  
  "numberOfElements": 3,  
  "first": true,  
  "last": false,  
  "empty": false  
  // pageable y sort completos: ver el JSON íntegro en los apuntes web  
}
```

🔗 *JSON completo: apuntes web — Tema 1, «Consultas dinámicas: Specifications y paginación»*

Los campos de Page, uno a uno

Campo	Qué es
content	Los elementos de esta página
totalElements	Filas que cumplen la consulta en total, no solo las de esta página
totalPages	Páginas necesarias para recorrerlas todas ($\text{totalElements} / \text{size}$, redondeado hacia arriba)
number	La página que estás viendo ahora mismo (empieza en 0)
size	El tamaño de página que has pedido
numberOfElements	Cuántos elementos trae esta página en concreto — distinto de size en la última página
first / last	Si esta es la primera o la última página

En la práctica, solo importa un puñado

Solo un puñado de estos campos importa de verdad — `content`, `totalElements`, `totalPages`, `number` — el resto es información derivada de esos mismos datos, útil de vez en cuando pero no algo que vayas a leer en cada petición.

Si el campo de sort no existe, de momento da 500

⚠ Un 500 que en realidad es culpa del cliente

Pide, por ejemplo, `sort=noExiste,asc` y la aplicación responde con un 500 Internal Server Error, no con un 400 — un error de entrada del cliente disfrazado de fallo del servidor. Se resolverá más adelante, en Programación de Servicios y Procesos, con `GlobalExceptionHandler`.

La solución completa, sin ningún filtro de por medio

Ni siquiera hace falta JpaSpecificationExecutor para esto: `findAll(Pageable)` ya viene incluido en el `JpaRepository` de siempre.

```
@Transactional(readonly = true)
public Page<LibroResponseDTO> findAllPaginated(Pageable pageable) {
    return libroRepository.findAll(pageable)
        .map(this::mapToDTO);
}
```

El controller paginado

```
@GetMapping
public ResponseEntity<Page<LibroResponseDTO>> getAll (@PageableDefault(size = 20) Pageable
pageable) {
    return ResponseEntity.ok(libroService.findAllPaginated(pageable));
}
```

Un tamaño por defecto, y un cambio de forma

`@PageableDefault(size = 20)` fija un tamaño de página por defecto si el cliente no especifica ninguno. Fíjate en que este `getAll()` ya no devuelve `List`, sino `Page` — ese cambio de forma en la respuesta es justo lo que vas a tener en cuenta al combinarlo con los filtros.

Todo junto: Specifications + paginación

En tu proyecto real no vas a mantener tres endpoints separados — vas a tener uno solo, que hace ambas cosas a la vez. Esto es lo que cambia respecto a las dos versiones aisladas de arriba:

Lo que cambia en la versión combinada

Pieza	Solo Specifications	Solo paginación	Versión combinada
Repositorio	JpaSpecificationExecutor	Ninguno extra	JpaSpecificationExecutor
Service, devuelve	List<LibroResponseDTO>	Page<LibroResponseDTO>	Page<LibroResponseDTO>
Service, recibe	LibroFiltroDTO filtro	Pageable pageable	Los dos
findAll(...)	con spec	con pageable	con spec y pageable
Controller	@ModelAttribute filtro	@PageableDefault pageable	Los dos a la vez

El service, con la misma Specification y el Pageable

```
@Transactional(readOnly = true)
public Page<LibroResponseDTO> findAllPaginated(LibroFiltroDTO filtro, Pageable pageable) {
    Specification<Libro> spec = Specification
        .where(LibroSpecifications.tituloContiene(filtro.titulo()))
        .and(LibroSpecifications.precioMayorOIgualA(filtro.precioMin()))
        .and(LibroSpecifications.precioMenorOIgualA(filtro.precioMax()))
        .and(LibroSpecifications.perteneceAEditorial(filtro.editorialId()));

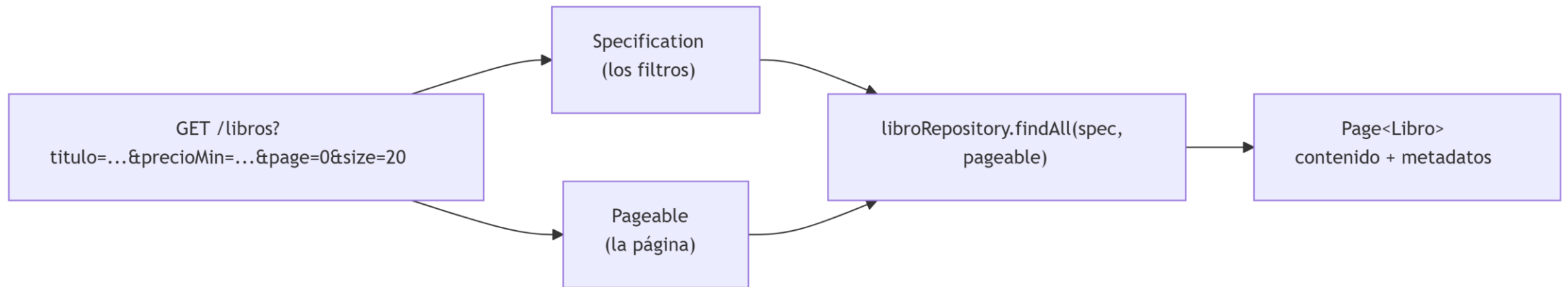
    return libroRepository.findAll(spec, pageable)
        .map(this::mapToDTO);
}
```

El controller combina los dos parámetros

```
@GetMapping
public ResponseEntity<Page<LibroResponseDTO>> getAll(
    @ModelAttribute LibroFiltroDTO filtro,
    @PageableDefault(size = 20) Pageable pageable
) {
    return ResponseEntity.ok(libroService.findAllPaginated(filtro, pageable));
}
```

`libroRepository.findAll(spec, pageable)` aplica la Specification combinada y, de paso, trocea el resultado en la página pedida — una sola llamada resuelve los dos problemas del apartado.

Exactamente lo que mostraba el diagrama del principio



Para que Swagger UI lo documente bien

Si pruebas este endpoint desde Swagger UI, puede que el formulario no te muestre un campo por cada filtro por separado — en su lugar, un único cuadro de texto con JSON. Pasa porque springdoc-openapi no siempre "explota" un objeto `@ModelAttribute` (ni siempre `Pageable`) en parámetros de query independientes.

```
@GetMapping
public ResponseEntity<Page<LibroResponseDTO>> getAll(
    @ParameterObject @ModelAttribute LibroFiltroDTO filtro,
    @ParameterObject @PageableDefault(size = 20) Pageable pageable
) {
    return ResponseEntity.ok(libroService.findAllPaginated(filtro, pageable));
}
```

Ideas clave

- `findById + save()` recuperan y modifican un objeto conocido por su id — no sirven cuando el usuario busca por criterios variables.
- Problema A (los filtros no escalan) y problema B (no se puede devolver todo de golpe) son problemas distintos que comparten el mismo listado como escenario.
- Una `Specification` construye la consulta como piezas combinables en tiempo de ejecución; `Specification.unrestricted()` es la pieza "vacía" para un filtro null.
- La paginación trocea el resultado en páginas; `Pageable` modela la petición, `Page<T>` la respuesta. Las páginas se numeran desde 0, y por debajo es `LIMIT/OFFSET` de SQL.

Ideas clave (cont.)

- JpaSpecificationExecutor habilita Specifications sobre un repositorio; findAll(spec), findAll(pageable) y findAll(spec, pageable) son tres sobrecargas del mismo método.
- @ParameterObject (de springdoc-openapi) hace que Swagger UI muestre un campo por cada atributo de filtro y de pageable, en vez de un único cuadro con JSON.